

MANUAL DE USUARIO

Compilador e IDE Multi-Lenguaje: Zetariano, Pig Latin y Y

Curso: Compiladores 2

Autor: Elmer Miguel

Proyecto: ContactoExtraterrestre

1. INTRODUCCIÓN

Bienvenido al manual de usuario del **Compilador e IDE Multi-Lenguaje**. Esta herramienta está diseñada para escribir, analizar, compilar y ejecutar programas interoperables escritos en tres lenguajes distintos:

1. **Pig Latin (.pig)**: Lenguaje imperativo orquestador que contiene la lógica principal del programa.
2. **Y? (.y)**: Lenguaje estructurado tipo Python con soporte de funciones, estructuras de datos (`struct`) y arreglos.
3. **Zetariano (.z)**: Lenguaje orientado a objetos con clases, atributos, métodos y constructores instanciados en memoria dinámica.

El sistema cuenta con una **interfaz gráfica moderna (IDE)** y una **interfaz de línea de comandos (CLI)**, integrando un compilador nativo que genera código intermedio de tres direcciones (C3D), código C estándar y un binario ejecutable mediante GCC.

2. REQUISITOS DEL SISTEMA

Para ejecutar la aplicación se requiere tener instalado en el sistema:

- **Java Development Kit (JDK)**: Versión 21 o superior.
- **Apache Maven**: Versión 3.8 o superior.
- **GCC (GNU Compiler Collection)**: Disponible en el `PATH` para la compilación de ejecutables nativos.
- **Sistema Operativo**: Linux (Ubuntu, Debian, Fedora, Arch) o cualquier distribución compatible.

3. INICIO DE LA APLICACIÓN

3.1. Modo Interfaz Gráfica (IDE)

Para abrir el entorno visual con pestañas de edición, explorador y consola:

```
mvn compile exec:java -Dexec.mainClass="elmer.compi2.zetariano.App"
```

3.2. Modo Línea de Comandos (CLI)

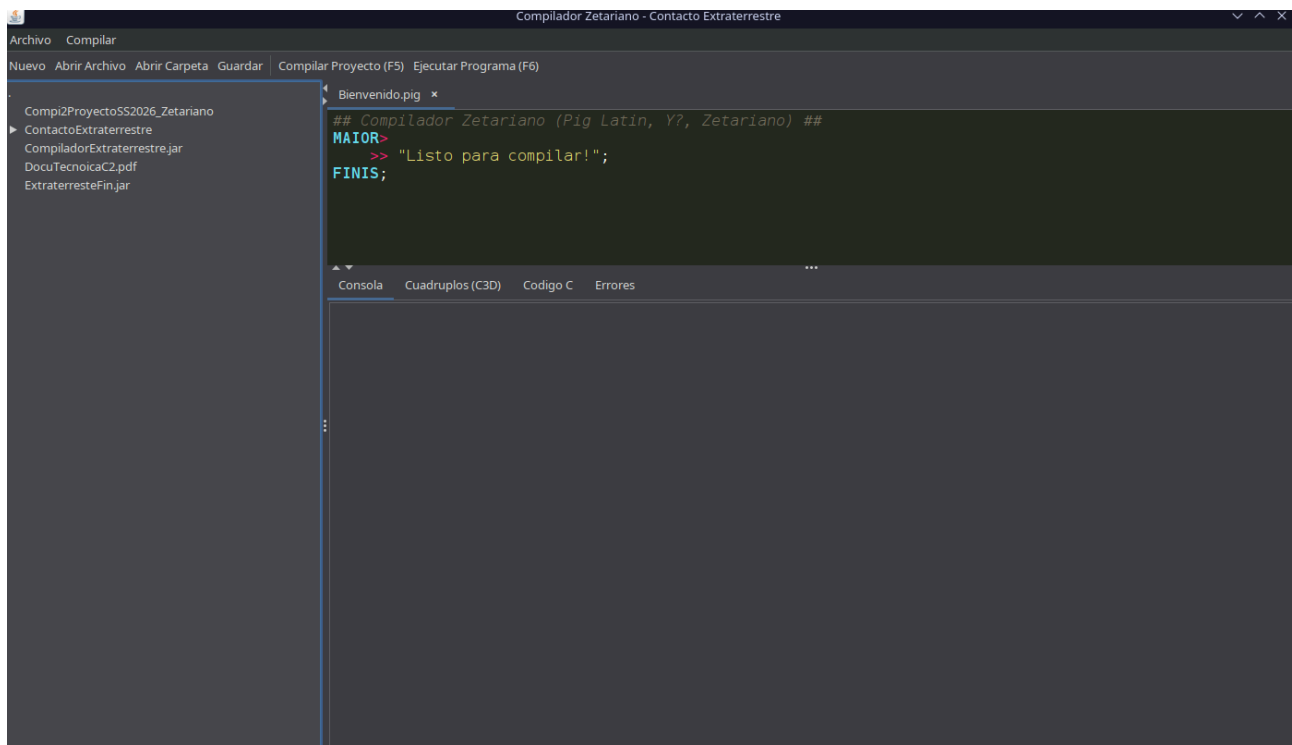
Para compilar y ejecutar directamente un archivo desde la terminal sin abrir la interfaz gráfica:

```
# Solo compilar
mvn compile exec:java -Dexec.mainClass="elmer.compi2.zetariano.App" -
Dexec.args="docs/Pruebas/Principal.pig"

# Compilar y ejecutar de inmediato el binario nativo
mvn compile exec:java -Dexec.mainClass="elmer.compi2.zetariano.App" -
Dexec.args="docs/Pruebas/Principal.pig --run"
```

4. DESCRIPCIÓN DE LA INTERFAZ GRÁFICA

Al iniciar la aplicación en modo gráfico, se presenta la ventana principal dividida en áreas funcionales:



Componentes Principales:

1. Barra de Herramientas (Superior):

- **Nuevo:** Crea una pestaña vacía para escribir código.
- **Abrir:** Abre un diálogo para seleccionar archivos `.pig`, `.y` o `.z`.
- **Guardar:** Guarda los cambios del archivo actualmente activo.
- **Compilar:** Inicia el análisis multi-lenguaje, generación C3D y traducción a C.
- **Ejecutar:** Compila con GCC y corre el programa en una consola interactiva.
- **Limpiar:** Limpia las salidas, consolas y tablas de resultados.

2. Explorador de Archivos (Panel Izquierdo):

- Muestra la estructura de carpetas de tu proyecto (docs/Pruebas/ , etc.).
- Al hacer doble clic en cualquier archivo, se abre en una nueva pestaña en el editor.

3. Editor de Código con Pestañas (Panel Central):

- Permite editar múltiples archivos simultáneamente.
- Resaltado de sintaxis dinámico adaptado a cada lenguaje (Pig Latin, Y?, Zetariano).
- Numeración de líneas a la izquierda.

4. Panel de Resultados (Panel Inferior):

- Pestañas especializadas para inspeccionar todas las fases de la compilación.

5. GUÍA PASO A PASO: CÓMO COMPILAR Y EJECUTAR UN PROGRAMA

A continuación se muestra el flujo de trabajo típico para compilar un proyecto políglota:

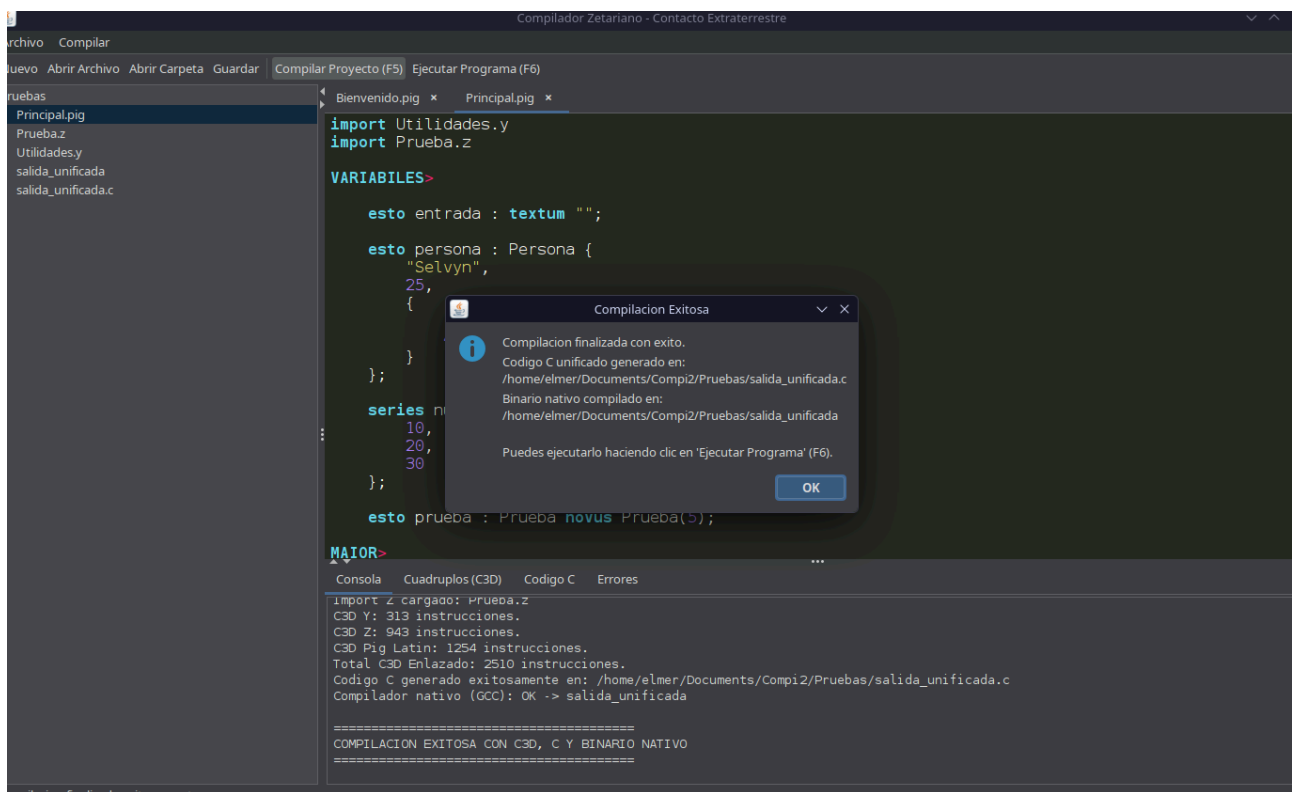
Paso 1: Abrir el archivo principal

1. En el **Explorador de Archivos**, navega hasta la carpeta `docs/Pruebas/`.
2. Haz doble clic en `Principal.pig`.
3. Verás cómo se abre el archivo en el editor con su resaltado de sintaxis correspondiente.
4. Observa las directivas de importación al inicio:

```
@import "Utilidades.y"
@import "Prueba.z"
```

Paso 2: Ejecutar la Compilación

1. Haz clic en el botón **Ejecutar** (o en el botón **Compilar**).
2. El compilador automáticamente:
 - Leerá y parseará `Principal.pig`.
 - Resolverá e importará las funciones y estructuras de `Utilidades.y`.
 - Resolverá e importará las clases y métodos de `Prueba.z`.
 - Verificará la semántica y tipos de todo el conjunto.
 - Generará el código C3D enlazado.
 - Generará el archivo `salida_unificada.c`.
 - Invocará a GCC para generar el binario ejecutable y lo iniciará.



6. INSPECCIÓN DE RESULTADOS Y SALIDAS

El panel inferior cuenta con pestañas diseñadas para entender e inspeccionar el funcionamiento interno del compilador:

6.1. Pestaña: Cuádruplos (C3D)

Muestra todas las instrucciones intermedias de tres direcciones enlazadas en orden secuencial (`operador, arg1, arg2, resultado`):

Consola	Cuadрупlos (C3D)	Codigo C	Errores
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , 61 , - , H)			
(+ , H , 1 , H)			
(HEAP_SET , -1 , - , H)			
(+ , H , 1 , H)			
(PRINTLN_STRING, t136 , - , -)			
(FUNC_END , main_pig , - , -)			

6.2. Pestaña: Código C Generado

Muestra el código C unificado generado por el backend (`salida_unificada.c`), incluyendo la emulación del Stack (`stack[]`), Heap (`heap[]`), punteros `P` y `H`, y las funciones traducidas:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#include <ctype.h>
#include <errno.h>
#include <limits.h>

#define STACK_SIZE 100000
#define HEAP_SIZE 100000

double Stack[STACK_SIZE];
double Heap[HEAP_SIZE];

double P = 0;
double H = 0;

void runtime_print_string(double ptr) {
    int p = (int)ptr;
    while ((int)Heap[p] != -1) {
        putchar((char)((int)Heap[p]));
    }
}

```

6.3. Pestaña: Tabla de Errores

Si el código fuente contiene errores léxicos, sintácticos o semánticos (por ejemplo, tipos incompatibles o variables no declaradas), el compilador los reportará en una tabla organizada con:

- **Tipo de Error:** Léxico, Sintáctico o Semántico.
- **Línea y Columna:** Ubicación exacta del error en el código.
- **Mensaje:** Descripción clara del fallo.

Fase	Linea	Columna	Mensaje
SINTACTICO	4	0	extraneous input 'asds' expecting {'im

7. RESOLUCIÓN DE PROBLEMAS FRECUENTES

Problema	Causa Posible	Solución
Error: GCC no encontrado	No está instalado GCC en tu sistema operativo Linux.	Ejecuta en terminal: <code>sudo apt update && sudo apt install build-essential gcc</code>
Error al importar archivo .y o .z	La ruta en <code>@import "archivo"</code> no coincide con la ubicación relativa.	Asegúrate de que el archivo importado esté en la misma carpeta que el archivo <code>.pig</code> o especifica la ruta correcta.
Error semántico: Tipos incompatibles	Se intentó asignar un tipo decimal a un entero o llamar una función con parámetros erróneos.	Revisa la pestaña <i>Errores</i> para identificar la línea exacta y corrige la signatura o variable.
Error: Command execution failed	No se compiló el proyecto previamente antes de ejecutar.	Ejecuta <code>mvn compile</code> en la raíz del proyecto para actualizar las clases compiladas.